

Android客户端分析

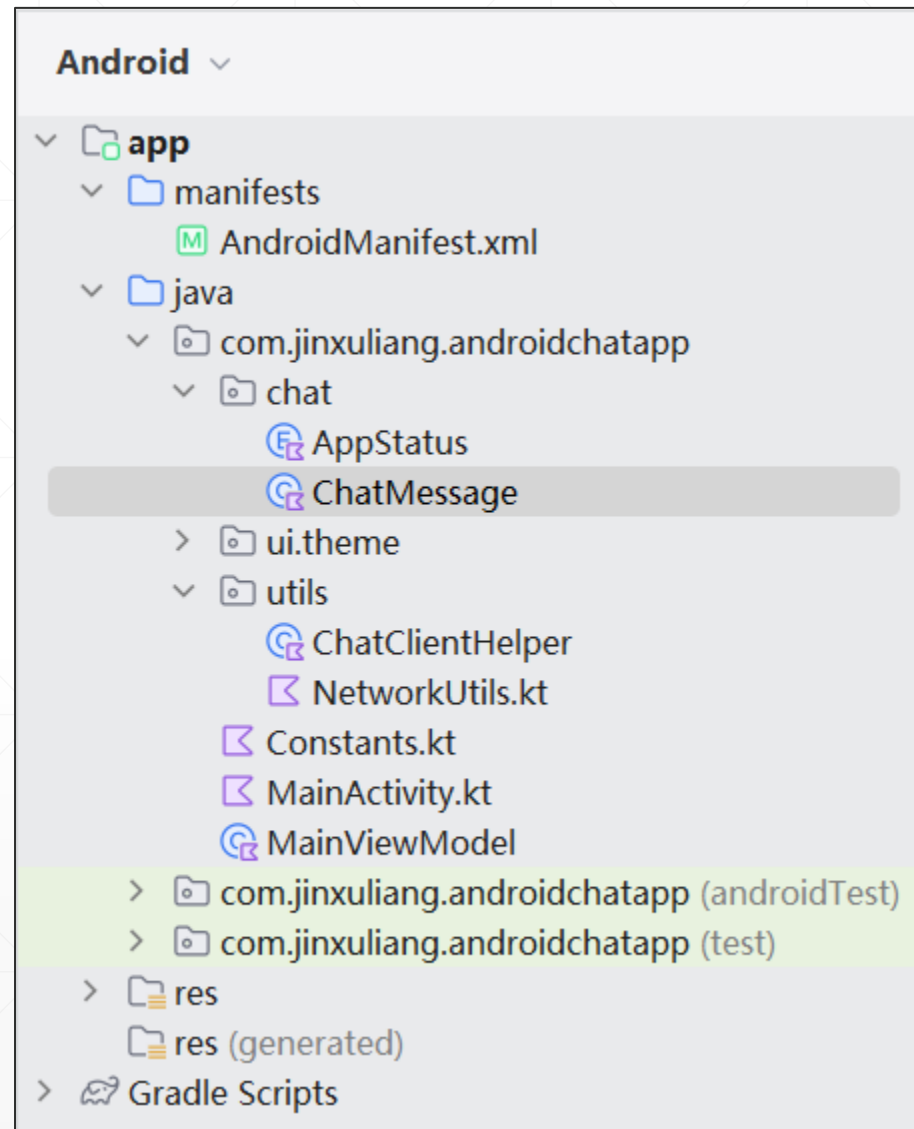
北京理工大学计算机学院
金旭亮

项目结构

示例使用Jetpack Compose开发

网络请求，使用Kotlin协程执行

主要功能放在ViewModel中，供
Activity绑定显示



辅助类型

```
//App状态
enum class AppStatus {
    NORMAL,
    SERVER_CONNECTED,
    MESSAGE_SEND
}
```

当状态改变时，UI界面应该有相应的变化，这个变化，是在ViewModel中引发的。

ChatMessage，代表聊天消息，为了方便传输与处理，将其序列化为JSON。

```
class ChatMessage(
    val userName: String,
    val message: String
) {
    companion object {
        fun toJson(message: ChatMessage): String {
            return Gson().toJson(message)
        }

        fun fromJson(json: String): ChatMessage {
            return Gson().fromJson(json, ChatMessage::class.java)
        }
    }
}
```

ViewModel-数据成员

```
class MainViewModel(app: Application) : AndroidViewModel(app) {  
    val serverIP = mutableStateOf(value: "192.168.1.5")  
    val userName = mutableStateOf(  
        UUID.randomUUID()  
        .toString().substringAfterLast(delimiter: '-')  
    )  
    val messages = mutableStateListOf("")  
    val appState = mutableStateOf(AppStatus.NORMAL)  
    private var socket: Socket? = null  
    //接收消息  
    private var reader: BufferedReader? = null  
    //发送消息  
    private var writer: BufferedWriter? = null
```

ViewModel1-连接服务器

```
fun connectServer() {  
    viewModelScope.launch(Dispatchers.IO) { this: CoroutineScope  
        socket = Socket(serverIP.value, DEFAULT_SERVER_PORT)  
        // 创建IO流  
        reader = BufferedReader(  
            InputStreamReader(socket!!.getInputStream())  
        );  
        writer = BufferedWriter(  
            OutputStreamWriter(socket!!.getOutputStream())  
        );  
        //_info.value = "成功连接服务器"  
        appState.value = AppStatus.SERVER_CONNECTED  
        var line = reader!!.readLine()  
        while (line != null) {  
            messages.add(line)  
            Log.d(TAG, line)  
            line = reader!!.readLine()  
        }  
    }  
}
```

ViewModel-发送消息

```
val message = mutableStateOf( value: "")

fun sendMessage() {
    Log.d(TAG, msg: "writer==null? ${writer}")
    val sb = StringBuffer()

    sb.append("writer==null?" + (writer == null))
    sb.append("mssage==null?" + (message.value == null))
    viewModelScope.launch(Dispatchers.IO) { this: CoroutineScope
        val chatMessage = ChatMessage(userName.value, message = message.value)
        writer!!.write( str: ChatMessage.toJson(chatMessage) + "\n")
        writer!!.flush()
        appState.value = AppStatus.MESSAGE_SEND
        message.value = ""
    }
}
```

UI界面

```
@Composable
fun MyChatApp() {
    val vm: MainViewModel = viewModel()
    MainContent(vm)
}
```

使用ViewModel作为状态容器，在各个组合函数中共享状态。

```
@Composable
fun MainContent(vm: MainViewModel) {
    //响应状态变化
    showStatus(state = vm.appState.value)
    Column(modifier = Modifier.fillMaxSize().padding(20.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Top
    ) { this: ColumnScope
        userNameUi(vm) //用户昵称相关UI
        Spacer(modifier = Modifier.height(20.dp))
        connectServerUi(vm) //连接服务器相关UI
        //空行
        Spacer(modifier = Modifier.height(20.dp))
        inputAndSendMessage(vm) //发送消息相关UI
        Spacer(modifier = Modifier.height(20.dp))
        messageListUi(vm) //消息列表UI
    }
}
```